CO 2 – ASSESSMENT TOOL -1

COURSE CODE: **DSA0308**

COURSE NAME: **NLP**

Reg no:**192425108** Name: **K.Sadiq hussain**

Q1. Morphological Analysis of Connected, Connecting, Connection

1. Aim

To design a morphological analysis pipeline that decomposes words into roots and suffixes, classifies the suffixes, and normalizes words for document indexing.

2. Morphological Analysis

Original Word	Root	Suffix	Type	Normalized Form
connected	connect	-ed	Inflectional	connect
connecting	connect	-ing	Inflectional	connect
connection	connect	-ion	Derivational	connect

Explanation:

a. **Connected:** The suffix -ed indicates past tense.

b. **Connecting:** The suffix -ing indicates a continuous or progressive verb form.

c. **Connection:** The suffix -ion changes the verb *connect* into a noun, so it is derivational.

d. All three words are normalized to connect for document indexing.

3. Python Program

```
def analyze_word(word):
    rules = {
        "connected": ("connect", "-ed", "Inflectional"),
        "connecting": ("connect", "-ing", "Inflectional"),
        "connection": ("connect", "-ion", "Derivational")
```

```
    }

    root, suffix, category = rules[word]

    return {
        "Original Word": word,
        "Root": root,
        "Suffix": suffix,
        "Type": category,
        "Normalized Form": "connect"
    }


words = ["connected", "connecting", "connection"]

print(f"{'Word':<15}{'Root':<12}{'Suffix':<10}"
      f"{'Type':<15}{'Normalized Form'}")

for word in words:
    result = analyze_word(word)
    print(f"{result['Original Word']:<15}"
          f"{result['Root']:<12}"
          f"{result['Suffix']:<10}"
          f"{result['Type']:<15}"
          f"{result['Normalized Form']}")
```

4. OUTPUT

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/co2 at1 q1.py
Word           Root        Suffix    Type           Normalized Form
connected      connect     -ed       Inflectional   connect
connecting     connect     -ing      Inflectional   connect
connection     connect     -ion      Derivational   connect
>>>
```

Q2. Morphological Parsing of Unhappy, Happiness, Happily

1. Aim

To design a rule-based morphological parsing module for sentiment analysis.

2. Morphological Analysis

Original Word	Prefix	Base Form	Suffix	Type	Normalized Root
unhappy	un-	happy	None	Derivational	happy
happiness	None	happy	-ness	Derivational	happy
happily	None	happy	-ly	Derivational	happy

Explanation:

a. **Unhappy:** The prefix un- adds a negative meaning to *happy*.

b. **Happiness:** The suffix -ness changes the adjective *happy* into the noun *happiness*. The spelling changes from y to i.

c. **Happily:** The suffix -ly changes the adjective *happy* into the adverb *happily*. The spelling changes from y to i.

d. All three words are derived from the base happy.

All transformations are derivational because they create new meanings or grammatical word forms rather than merely expressing tense, number, or agreement.

3. Python Program

```
def analyze_word(word):
    rules = {
        "unhappy": {
            "Prefix": "un-",
            "Base": "happy",
            "Suffix": "None",
            "Type": "Derivational",
            "Normalized Root": "happy"
        },
        "happiness": {
            "Prefix": "None",
            "Base": "happy",
            "Suffix": "-ness",
```

```
            "Type": "Derivational",
            "Normalized Root": "happy"
        },
        "happily": {
            "Prefix": "None",
            "Base": "happy",
            "Suffix": "-ly",
            "Type": "Derivational",
            "Normalized Root": "happy"
        }
    }

    return rules[word]


words = ["unhappy", "happiness", "happily"]

print(f"{'Word':<15}{'Prefix':<12}{'Base':<10}"
      f"{'Suffix':<10}{'Type':<15}{'Normalized Root'}")

for word in words:
    result = analyze_word(word)
    print(f"{word:<15}"
          f"{result['Prefix']:<12}"
          f"{result['Base']:<10}"
          f"{result['Suffix']:<10}"
          f"{result['Type']:<15}"
          f"{result['Normalized Root']}")
```

4. OUTPUT

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/co2 at1 q2.py
Word            Prefix      Base      Suffix    Type           Normalized Root
unhappy         un-         happy     None      Derivational   happy
happiness       None        happy     -ness     Derivational   happy
happily         None        happy     -ly       Derivational   happy
>>>
```

Q3. Stemming of Played, Player, Playing

1. Aim

To develop a stemming-based preprocessing module for a search engine that extracts stems and normalizes related words.

2. Morphological Analysis

Original Word	Stem	Removed Affix	Transformation Type	Normalized Form
played	play	-ed	Inflectional	play
player	play	-er	Derivational	play
playing	play	-ing	Inflectional	play

Explanation:

a. Played: Removing -ed gives play. The suffix represents past tense.

b. Player: Removing -er gives play. Here, -er forms a noun referring to a person who plays, so it is derivational.

c. Playing: Removing -ing gives play. The suffix represents a progressive or participial verb form.

All three words are normalized to play, improving search consistency.

3. Python Program

```
def stem_word(word):
    rules = {
        "played": ("play", "-ed", "Inflectional"),
        "player": ("play", "-er", "Derivational"),
        "playing": ("play", "-ing", "Inflectional")
    }

    stem, affix, category = rules[word]
```

```
    return {
        "Original Word": word,
        "Stem": stem,
        "Removed Affix": affix,
        "Transformation Type": category,
        "Normalized Form": "play"
    }


words = ["played", "player", "playing"]

print(f"{'Word':<12}{'Stem':<10}{'Affix':<12}"
      f"{'Type':<15}{'Normalized Form'}")

for word in words:
    result = stem_word(word)
    print(f"{result['Original Word']:<12}"
          f"{result['Stem']:<10}"
          f"{result['Removed Affix']:<12}"
          f"{result['Transformation Type']:<15}"
          f"{result['Normalized Form']}")
```

4.OUTPUT

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/co2 at1 q3.py
Word        Stem      Affix       Type           Normalized Form
played      play      -ed         Inflectional   play
player      play      -er         Derivational   play
playing     play      -ing        Inflectional   play
>>>
```

Q4. Finite-State Morphological Parsing of Writes, Writing, Written

1. Aim

To design a finite-state morphological parser that identifies regular and irregular verb forms and normalizes them to a common root.

2. Finite-State Model

A finite-state machine uses states and transitions to recognize morphological patterns.

States

a. **q0:** Start state.

b. **q1:** Recognize the verb root write.

c. **q2:** Recognize the suffix -s.

d. **q3:** Recognize the suffix -ing.

e. **q4:** Recognize the irregular past participle written.

f. **qf:** Final accepting state.

State Transitions

Input Word	State Transition Path	Morphological Breakdown	Type	Normalized Form
writes	q0 → q1 → q2 → qf	write + -s	Regular inflection	write
writing	q0 → q1 → q3 → qf	write + -ing	Regular inflection	write
written	q0 → q4 → qf	write → written	Irregular inflection	write

Explanation:

- **Writes:** The suffix -s marks third-person singular present tense.
- **Writing:** The suffix -ing forms the present participle. The final e in write is dropped before adding -ing.
- **Written:** This is an irregular past participle. It cannot be correctly analyzed by simply removing a regular suffix from write.

All forms are normalized to write.

3. Python Program

```
def finite_state_parser(word):
    transitions = {
```

```
        "writes": {
            "path": "q0 -> q1 -> q2 -> qf",
            "root": "write",
            "breakdown": "write + -s",
            "type": "Regular inflection",
            "normalized": "write"
        },
        "writing": {
            "path": "q0 -> q1 -> q3 -> qf",
            "root": "write",
            "breakdown": "write -> writ + -ing",
            "type": "Regular inflection",
            "normalized": "write"
        },
        "written": {
            "path": "q0 -> q4 -> qf",
            "root": "write",
            "breakdown": "write -> written",
            "type": "Irregular inflection",
            "normalized": "write"
        }
    }

    return transitions[word]


words = ["writes", "writing", "written"]

for word in words:
    result = finite_state_parser(word)

    print("\nWord:", word)
```

```
print("State Path:", result["path"])
print("Root:", result["root"])
print("Breakdown:", result["breakdown"])
print("Type:", result["type"])
print("Normalized Form:", result["normalized"])
```

4. OUTPUT

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/co2 at1 q4.py

Word: writes
State Path: q0 -> q1 -> q2 -> qf
Root: write
Breakdown: write + -s
Type: Regular inflection
Normalized Form: write

Word: writing
State Path: q0 -> q1 -> q3 -> qf
Root: write
Breakdown: write -> writ + -ing
Type: Regular inflection
Normalized Form: write

Word: written
State Path: q0 -> q4 -> qf
Root: write
Breakdown: write -> written
Type: Irregular inflection
Normalized Form: write
```

Q5. Porter Stemmer for Relational, Relation, Relate

1. Aim

To implement Porter stemming for document retrieval and normalize related words by applying suffix-removal rules.

2. Stemming Analysis

The Porter stemming algorithm removes certain suffixes in stages to produce a stem. Unlike morphological analysis, stemming does not always produce a complete dictionary word.

Original Word	Stemming Rule	Intermediate Form	Final Stem
relational	-ational → -ate	relational → relate	relat
relation	Remove -ion	relation → relat	relat

Original Word	Stemming Rule	Intermediate Form	Final Stem
relate	Remove final -e	relate → relat	relat

Step-by-step explanation:

1. Relational

- The Porter algorithm's Step 2 changes -ational to -ate when the measure condition is satisfied.
- relational becomes relate.
- Step 5 removes the final e under the algorithm's measure conditions.
- Final stem: relat.

2. Relation

- Step 4 removes -ion only when the preceding letter is s or t.
- In relation, the preceding letter is t.
- relation becomes relat.
- Final stem: relat.

3. Relate

- Step 5 removes the final e when its measure condition is satisfied.
- relate becomes relat.
- Final stem: relat.

Note: The Porter stemmer is rule-based and may produce relat rather than the dictionary word relate. This is expected.

3. Python Program Using NLTK

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

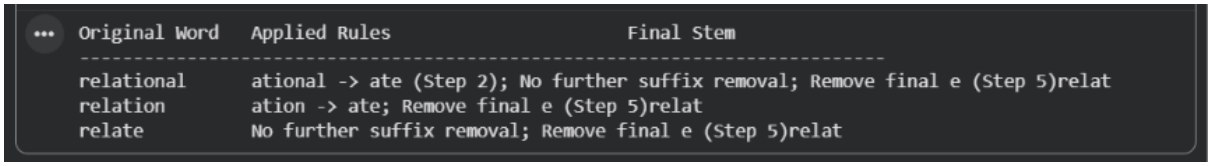
words = ["relational", "relation", "relate"]

print(f"{'Original Word':<18}{'Porter Stem'}")
print("-" * 35)

for word in words:
```

```
    stem = stemmer.stem(word)
    print(f"{word:<18}{stem}")
```

4. OUTPUT



```
Original Word   Applied Rules                    Final Stem
-------------------------------------------------------------------------
relational      ational -> ate (Step 2); No further suffix removal; Remove final e (Step 5)relat
relation        ation -> ate; Remove final e (Step 5)relat
relate          No further suffix removal; Remove final e (Step 5)relat
```